iRacing Setup Machine

High-Level Technical Specification

Version 3.1 • March 2026

Architecture, Feature Reference & Integration Guide

39	8	18	3
Core Modules	UI Mixins	Backend Files	Tiers

1. Executive Overview

The **iRacing Setup Machine** is a full-stack commercial platform combining a professional desktop telemetry application with a cloud SaaS backend. It is purpose-built for competitive iRacing drivers and teams who need data-driven setup advice, AI-powered coaching, and collaborative tools — all in a single integrated product.

Platform Pillars

Desktop App	Python 3.12 + CustomTkinter	A dark-mode, high-DPI telemetry workstation that runs entirely offline. Parses raw .ibt binary files, runs 15+ analysis engines, renders 15+ chart types, and streams AI advice — all locally.
SaaS Backend	FastAPI + PostgreSQL + Redis	A production-ready cloud API providing session storage, async job processing, Stripe billing, JWT auth with 2FA, team collaboration, leaderboards, and server-side AI calls.
AI Layer	Anthropic Claude API (Haiku + Sonnet)	Three distinct AI interaction modes: recommendations (setup-focused), Q&A; chat (natural language), and the Telemetry Agent (tool-use, queries raw channels locally).
Race Engineer Persona	Steven (persistent profile)	An AI persona that accumulates a driver profile across sessions using exponential weighted averages. Activates after 3 sessions and provides personalised, data-grounded coaching.

2. System Architecture

2.1 High-Level Component Diagram

The system is divided into three deployment targets that communicate over HTTPS/WSS:

Layer	Technology	Deployment	Key Responsibilities
Desktop App	Python 3.12, CustomTkinter, Matplotlib, NumPy	User machine (Windows/macOS)	IBT parsing, analysis, charts, AI streaming, local tool execution, OBS overlay
Cloud API	FastAPI, SQLAlchemy 2.0 async, Pydantic v2	Docker (Uvicorn + Gunicorn)	Auth, session processing, AI proxy, billing, team management, leaderboard, WebSocket
Background Worker	ARQ (async Redis queue), same Python env	Docker (separate container)	Heavy telemetry processing, PDF generation, IBT cleanup cron jobs
Data Stores	PostgreSQL 16, Redis 7, S3/Cloudflare R2	Managed cloud services	Persistent data, rate limiting, job queues, object storage (IBT, reports, PDFs)

2.2 Desktop App Class Hierarchy

The desktop application uses Python's multiple-inheritance mixin pattern. The **App** class inherits from specialised tab mixins, keeping each feature domain self-contained:

```
App(IRacingTabMixin, TelemetryTabMixin, CornersTabMixin, StintTabMixin, ctk.CTk)
```

Tab content is built **lazily** — each tab's UI is constructed only on first visit (_tab_builders dict + _built_tabs set). This keeps startup time under 1 second regardless of how many tabs exist.

2.3 Key Data Flows

IBT Load & Analysis

User opens .ibt → IBTParser reads binary (header + channel arrays) → AnalysisEngine produces AnalysisReport → SectorAnalyzer, CornerAnalyzer, DrivingStyleAnalyzer, ConsistencyScorer run in background thread → UI refreshes all stale tabs on completion.

AI Recommendations

User clicks Get Recommendations → API key fetched from OS keyring → If Steven persona active: RaceEngineer.get_recommendations_stream() called; otherwise get_ai_recommendations_stream() used → Comprehensive prompt built (report + setup + style + sector + corner data) → Claude API streamed chunk-by-chunk into CTkTextbox.

Telemetry Agent

User asks natural language question → ask_telemetry_agent() called locally → Claude tool-use loop: up to 6 rounds of list_channels / get_channel_stats / get_channel_slice / compare_laps / get_lap_times tool calls → All tool execution is local (no raw data sent to API) → streamed answer.

Cloud Session Upload

cloud_sync.upload_session(ibt_path) → POST /sessions → IBT stored to S3 → TelemetrySession created (status=pending) → ARQ job enqueued → Worker downloads IBT, runs full pipeline → Report JSON uploaded to S3 → WebSocket push (status=complete) → Desktop receives notification via watch_session_status() WebSocket client.

Stripe Billing

User clicks Upgrade → POST /billing/checkout → Stripe Checkout session created → User redirected to Stripe-hosted page → Payment completes → Stripe webhook POST /billing/webhook → Subscription updated in DB → license_state.tier updated on next token refresh.

3. Core Analysis Modules

All analysis logic lives in **files/core/** and is shared between the desktop app and the ARQ background worker. Modules are stateless — each returns a typed dataclass report from a pure function call.

Module	Key Exports	Description
ibt_parser.py	IBTParser, TelemetryData, load_demo_data	Parses iRacing .ibt binary telemetry. Reads 144-byte file header, variable metadata block, and sample data into NumPy arrays. Reconstructs lap boundaries from SessionNum + LapCompleted channels. Returns TelemetryData with _channels dict, lap_times, lap_boundaries, tick_rate, session_info.
analysis_engine.py	AnalysisEngine, AnalysisReport, Issue, Severity	Primary analysis pass. Evaluates tire temperatures/pressures, brake balance, suspension bottoming, fuel usage, RPM limiter hits, grip utilisation, and corner-phase balance. Issues classified as CRITICAL / WARNING / INFO with actionable recommendations. Configurable outlier threshold for lap filtering.
advanced_analysis.py	SectorAnalyzer, BestLapAnalyzer, StintAnalyzer, FuelStrategyAnalyzer	Sector analysis divides each lap into N equal sectors and computes per-sector times, speeds, lateral G, and tire temps. BestLapAnalyzer builds theoretical best by combining each session's fastest sector. StintAnalyzer models tire degradation rate (s/lap). FuelStrategyAnalyzer projects fuel loads for target stints.
corner_analysis.py	CornerAnalyzer, LapDeltaAnalyzer, CornerData	Identifies brake zones from brake channel edges to build a corner list. Per-corner: brake point %, apex speed, exit speed, brake pressure avg, throttle exit avg, lateral G peak, per-lap consistency. LapDeltaAnalyzer builds cumulative time delta trace (dist_pct vs delta_s) vs. a reference lap — used in PDF charts and OBS overlay.
driving_style.py	DrivingStyleAnalyzer, DriverStyleReport	Distinguishes driver technique issues from setup problems. Detects: trail-braking percentage, throttle-on point, steering reversals (nervousness), oversteer/understeer events (balance verdict), coast time %, throttle smoothness. Returns 0–100 scores per trait.
consistency_score.py	compute_consistency, ConsistencyBreakdown	Composite 0–100 consistency rating combining lap time variance (CV), sector repeatability, corner-entry speed std, and brake point std. Returns component scores, worst lap/sector/corner indices, letter grade A+–F.
session_quality.py	IncidentDetector, TrackEvolutionAnalyzer	IncidentDetector flags laps with abnormal G-spikes or LapDistPct teleports as incident laps to exclude from averages. TrackEvolutionAnalyzer separates genuine driver improvement from track rubber-in by fitting a linear regression to consecutive lap times.
brake_analysis.py	analyze_braking, BrakeAnalysisReport	Deep brake zone analysis: maximum brake pressure per zone, brake-release profile (threshold to trail-brake to release), lock detection (speed drop + wheel speed vs. ground speed divergence if available), per-zone consistency.
tire_wear_prediction.py	predict_tire_wear, TireWearPrediction	Models tire degradation using linear regression on rolling lap-time deltas. Estimates deg_rate (s/lap), cliff lap (where grip falls sharply), and per-corner deg rates from tire temperature trends.

ml_predictor.py	LapTimePredictor, SessionFeatures	NumPy Ridge Regression predictor (no external ML dependency). Trains on SessionFeatures (balance score, tire temps, fuel/lap, air/track temps) from loaded sessions. Returns PredictionResult with predicted lap time, confidence, feature importance dict, and a plain-English insight.
setup_impact.py	predict_impact, ImpactReport, compute_sensitivity_matrix	Physics-model setup change predictor. Maps parameter adjustments (wing angle, spring stiffness, ARB, brake bias, tire pressure) to expected effects on lap time delta, balance shift, and tire wear. Confidence scores indicate reliability per parameter.
setup_optimizer.py	optimize_setup, OptimizationTarget, OptimizationResult	Iterative setup optimiser that combines predict_impact with a grid-search or gradient-descent approach over adjustable parameters. Returns ranked OptimizationResult list with expected lap time gains and balance impacts.
setup_recommend.py	find_sto_files, score_sto_matches, build_checklist	Scans the user's iRacing setups folder for matching .sto files by car name similarity (SequenceMatcher ≥ 0.6). Scores and ranks candidates. Builds a ChecklistItem list with exact garage label, delta, unit, and current vs. recommended values for in-game application.
setup_parser.py	SetupParser, ParsedSetup, SetupExporter, StoWriter, SetupDiffer	Parses iRacing .htm setup exports (HTML tables) into ParsedSetup with sections and a flat key→value dict. SetupExporter writes back to .htm. StoWriter generates native iRacing .sto format (YAML-like text, placed in ~/Documents/iRacing/setups//). SetupDiffer returns changed parameters.
tech_inspector.py	validate_setup, clamp_to_legal, get_bounds	Enforces per-car-class legal parameter bounds (GT3, GT4, GTP, LMP2, Formula, Porsche Cup, TCR, etc.). validate_setup returns TechIssue list with out-of-range parameters. clamp_to_legal auto-corrects to nearest legal value.
racing_line.py	reconstruct_racing_line, speed_colormap	Reconstructs track outline and racing line from LapDistPct + lateral acceleration (or GPS if available). Returns XY coordinate arrays for track map rendering. speed_colormap generates a per-point colour array for heatmap visualisation.
gg_diagram.py	analyze_gg_per_corner, GGReport	Builds lateral vs. longitudinal G scatter plots per corner. Computes combined G utilisation (available grip fraction) and identifies under-utilised grip areas.
iracing_api.py	IRacingAPIClient, get_client, MemberSummary, RaceResult, BestLap	Full iRacing Member Data API client. Auth: base64(SHA-256(password + email.lower())). Most endpoints return {link: s3_url} — client follows redirect automatically. Converts lap times from tenths-of-milliseconds (÷10,000). Thread-safe with Lock. Returns typed dataclasses for all response types.
race_engineer.py	RaceEngineer, DriverProfile, DriverProfileManager	Persistent DriverProfile stored at ~/.iracing_engineer_profile.json. DriverProfileManager.update_from_session() applies EWA (α=0.20) to style traits, balance scores, and corner weakness records. RaceEngineer('Steven') activates after 3 sessions, injects profile context into Claude system prompt for personalised recommendations, Q&A;, and evolution summaries.
telemetry_agent.py	ask_telemetry_agent, _TOOL_SCHEMAS	Claude tool-use loop with 5 local tools: list_channels, get_lap_times, get_channel_stats, get_channel_slice, compare_laps. Up to 6 agentic rounds. All data processing is local — only the question + tool results leave the machine. Streams final natural-language answer back to the caller.

live_telemetry.py	LiveTelemetryMonitor, LiveSample	Polls iRacing shared memory (irsdk) at configurable Hz (default 10 Hz). Delivers LiveSample snapshots (speed, RPM, gear, throttle, brake, fuel, tire temps/pressures, lap times) via registered callbacks. Gracefully handles missing pyirsdk package.
cloud_sync.py	watch_session_status, submit_leaderboard, stream_ai_recommendations	Desktop cloud sync client. Uses WebSocket (websocket-client) for session status watching with HTTP polling fallback. Leaderboard submission/query. SSE streaming for AI calls. Background thread uploads with progress callbacks. Token refresh with automatic retry on 401.
config.py	get_api_key, set_api_key, get_iracing_credentials	API key and iRacing credentials stored exclusively in OS credential store (keyring package). Config file never contains plaintext secrets. Auto-migrates legacy plaintext keys to keyring on first load.
license.py	LicenseState, TIER_FEATURES, require_feature	Singleton license state disk-cached for offline operation. has_feature(feature) gates all premium UI. require_feature() shows a CTkToplevel upgrade prompt with CTA when a feature is missing.

4. Desktop User Interface

The desktop UI is built with **CustomTkinter** (dark mode, blue accent). All chart rendering uses **matplotlib + FigureCanvasTkAgg** embedded in custom EmbedChart frames. Tabs are lazily built on first visit to keep startup under 1 second.

4.1 Tab Inventory

Tab	Contents
Dashboard	Session KPIs, issue heat map, balance chart, tire heatmap, history sparklines, onboarding banner
Telemetry	15+ chart types (Speed+Brake+Throttle, tire temps/pressures, G-G, suspension, RPM/gear, lap overlay, reference delta, racing line, sector map, brake trace)
Issues	Categorised issue cards (CRITICAL / WARNING / INFO) with per-issue recommendation and fix guide
Driver	Driving style scores, oversteer/understeer verdict, technique breakdown, event log
Sectors	Per-sector time/speed/G comparison, theoretical best lap, sector consistency chart
Corners	Per-corner metric table, brake point / apex / exit analysis, coaching notes, consistency heat map
Stint & Tires	Tire degradation chart, cliff detection, fuel strategy calculator, pit stop planner
Lap Times	Lap time scatter, rolling average, outlier detection, lap selector with stint colouring
Brake Trace	Brake pressure trace, lock detection markers, trail-braking profile, zone consistency
Strategy	Multi-stint race strategy table, 0–5 stop options ranked by predicted total time
Trends	Session-over-session improvement tracking, ML lap time prediction, history chart
Impact	Setup change effect predictor, sensitivity matrix heat map, parameter adjustment sliders
Setup Files	Setup parser, parameter table, diff viewer, tech inspector, AI setup recommendation, Export .htm / .sto
AI Advisor	Streaming recommendations, Q&A; chat, Steven mode badge, ■ Evolution summary button
Agent	Natural language telemetry agent tab, example prompt chips, streaming chat log
Templates	Per-car-class setup templates, track-specific baseline parameters
History	Cross-session lap time trends, aggregated consistency, improvement velocity
Compare	Side-by-side session comparison, channel overlay for any two sessions
iRacing	iRating / SR trend charts, race results table, personal bests, best lap trend chart, community leaderboard

4.2 Peripheral UI Components

OBS Overlay (obs_overlay.py)

Frameless, always-on-top CTkToplevel HUD. Draggable. Shows: current lap time, Δ vs session best, speed/gear/RPM, throttle/brake fill-bars, 4-corner tyre temp grid (cold/optimal/warm/hot colour coding), live delta sparkline. Refreshes at 10 Hz from LiveTelemetryMonitor callbacks. Designed for OBS Window Capture.

Command Palette (Ctrl+K)

Fuzzy-search command launcher. Scoring: exact match (10) → prefix (8) → substring (6) → word-start (4) → difflib ratio > 0.4. Recently-used section (top 5, persisted in config). Keyboard Up/Down navigation. 20 result limit. Icons by category (■ file, ■ export, ■ ai, → navigate, ■ general).

Live Dashboard

Full-screen live telemetry window launched by ■ Live button. Shows real-time speed, RPM, gear, tire temps, G-forces. Requires pyirsdk for shared memory access.

Auth Dialog (auth_dialog.py)

Modal CTkToplevel for cloud account login/register. Mode switching between forms. AccountWidget header badge shows tier + sign-out. UsagePanel shows session/AI/storage progress bars with upgrade CTA.

5. SaaS Backend

5.1 API Routes

Endpoint	Min Tier	Description
POST /auth/register	FREE	Create account + free subscription + usage record. Returns token pair.
POST /auth/login	FREE	bcrypt verify → if TOTP enabled returns totp_token challenge; else returns access+refresh JWT pair.
POST /auth/totp/setup	FREE	Generate TOTP secret, return provisioning URI + base64 QR PNG data URL.
POST /auth/totp/enable	FREE	Verify first TOTP code, activate 2FA.
POST /auth/totp/verify	FREE	Step-2 TOTP login: verify code + totp_token → returns full token pair.
POST /auth/refresh	FREE	Rotate refresh token (old revoked, new issued). Returns new access+refresh pair.
GET /auth/me	FREE	Current user profile, tier, usage stats.
POST /sessions	FREE	Upload IBT (≤250 MB), create DB record, enqueue ARQ job. Returns session_id immediately.
GET /sessions/{id}/status	FREE	Poll processing status (pending/processing/complete/error).
GET /sessions/{id}/report	PRO	Full JSON analysis report (requires cloud_sync feature).
GET /sessions/{id}/channels	PRO	Downsampled channel data for any lap (up to 500 points).
GET /sessions/{id}/lap/{ref}/delta/{cmp}	PRO	LapDelta computation between two laps.
POST /sessions/{id}/export/pdf	PRO	Enqueue PDF generation job; returns presigned download URL on completion.
POST /sessions/{id}/share	PRO	Create share link (optional expiry). Returns token for public access.
POST /ai/recommendations	PRO	Stream SSE setup recommendations (Claude Haiku, checks AI quota).
POST /ai/chat	PRO	Multi-turn chat with 20-turn history window, streamed SSE.
POST /ai/session-note	PRO	Auto-generate 300-token session note.
POST /billing/checkout	FREE	Create Stripe Checkout session for pro/team upgrade. Returns checkout_url.
POST /billing/portal	PRO	Create Stripe Customer Portal link for subscription management.
POST /billing/webhook	FREE	Stripe webhook: handles subscription.created/updated/deleted, invoice.payment_failed.
POST /leaderboard/submit	FREE	Opt-in submission of best lap from verified session.
GET /leaderboard/times	FREE	Top lap times for car+track combo, paginated.
GET /leaderboard/my-standing	FREE	Caller's rank and percentile for a car+track.
POST /teams	TEAM	Create team (one per owner). Generates 12-char invite code.
POST /teams/{id}/members	TEAM	Invite member by email. Checks seat limit.
GET /admin/audit-logs	ADMIN	Paginated audit log with user/action/since filters.
GET /admin/stats	ADMIN	Platform stats: users by tier, session counts, AI calls, storage.

WS /ws/sessions/{id}	FREE	WebSocket: stream session processing updates (processing/complete/error).
WS /ws/user	FREE	WebSocket: stream all user events (session updates, quota warnings, team events).

5.2 Database Schema

PostgreSQL 16 via async SQLAlchemy 2.0. All tables use UUID primary keys. JSONB columns for flexible metadata (session_info, audit metadata, setup params).

Table	Key Columns	Notes
users	id, email, hashed_password, totp_secret, totp_enabled, is_admin, discord_id, google_id	OAuth-ready. 2FA via TOTP (pyotp). Email verification + password reset tokens.
subscriptions	user_id (unique), stripe_customer_id, stripe_subscription_id, tier, status, trial_end	One per user. tier: free|pro|team. status: active|trialing|past_due|cancelled|unpaid.
teams	id, name, owner_id, seats_purchased, invite_code	One team per owner. Members join via 12-char invite code. Coach/member roles.
team_members	team_id, user_id, role	Unique (team_id, user_id). role: owner|coach|member.
telemetry_sessions	user_id, storage_key, report_key, car_name, track_name, best_lap_s, status, job_id	status: pending→processing→complete→error. Dual S3 keys: raw IBT + processed JSON report.
setup_files	user_id, storage_key, car_name, parsed_params (JSONB)	parsed_params caches the flat key→value map to avoid re-parsing on diff requests.
share_links	resource_type, resource_id, token, expires_at, views	resource_type: session|setup. Public access via /shared/{token}. Hit counter.
usage_records	ai_calls_today, ai_calls_date, sessions_this_month, storage_bytes	One per user. Monthly session counter reset. Daily AI counter reset. Cumulative totals.
api_keys	user_id, key_prefix (8 chars), key_hash (SHA-256), expires_at	Full key shown once at creation. team tier only. Prefix for UI display.
refresh_tokens	user_id, token_hash, device_hint, expires_at, revoked	Rotation: old revoked when new issued. 30-day expiry.
audit_logs	user_id, action, resource_type, resource_id, ip_address, metadata (JSONB)	Immutable. Indexed on (user_id, created_at) and action. Never deleted.
leaderboard_entries	user_id, car_name, track_name, lap_time_s, display_name, is_verified	Opt-in. Unique per (user, car, track) — only kept if new time is faster.

5.3 Background Job Worker

ARQ (async Redis queue) powers background processing. Worker runs as a separate Docker container sharing the same Python environment and files/ directory.

Job	Trigger	Steps
process_telemetry_session	POST /sessions upload	Download IBT → IBTParser → AnalysisEngine → Sector/Corner/Style/Consistency analyzers → serialize to JSON → upload to S3 (report_key) → update DB status → WebSocket push.
generate_pdf_report_job	POST /sessions/{id}/export/pdf	Download report JSON → reconstruct minimal TelemetryData → generate_pdf_report() (ReportLab with track map + lap delta charts) → upload PDF to S3 → return presigned URL.

cleanup_expired_ibt	Cron (daily)	Delete raw .ibt files from S3 older than ibt_expiry_days setting. Preserves processed JSON reports. Logs bytes freed.

6. Feature & Tier Matrix

Feature	Free	Pro	Team
IBT Telemetry Analysis	✓	✓	✓
Basic Chart Types (Speed/Brake/Throttle)	✓	✓	✓
Issue Detection & Recommendations	✓	✓	✓
Track Map Visualisation	✓	✓	✓
Lap Delta Comparison	✓	✓	✓
iRacing Account Integration	✓	✓	✓
Community Leaderboard	✓	✓	✓
2FA / TOTP	✓	✓	✓
Advanced Telemetry Charts (15+ types)	—	✓	✓
Sector & Corner Analysis	—	✓	✓
Driving Style Analysis	—	✓	✓
Consistency Scoring	—	✓	✓
Tire Wear & Degradation	—	✓	✓
Stint & Race Strategy	—	✓	✓
ML Lap Time Predictor	—	✓	✓
Setup Optimiser & Impact Predictor	—	✓	✓
PDF Export	—	✓	✓
CSV / MoTeC Export	—	✓	✓
AI Setup Recommendations (200/day)	—	✓	✓
AI Q&A; Chat	—	✓	✓
Telemetry Agent (natural language)	—	✓	✓
Steven Persona (after 3 sessions)	—	✓	✓
Evolution AI Summary	—	✓	✓
OBS Overlay HUD	—	✓	✓
Cloud Sync (upload + report storage)	—	✓	✓
Sessions / month	10	Unlimited	Unlimited
AI calls / day	5	200	500
Cloud Storage	512 MB	50 GB	200 GB
Team Management & Shared Library	—	—	✓
Coach View (team sessions)	—	—	✓
Setup Sharing (team tier)	—	—	✓
API Key Access	—	—	✓
AI calls / day (team)	—	—	500

7. Security Architecture

Control	Implementation
Password Storage	bcrypt (passlib) with work factor 12. Never stored plaintext.
API Keys (backend)	Full key shown once. Only SHA-256 hash stored in DB. Prefix (8 chars) for UI display.
JWT Tokens	HS256-signed access tokens (15-min TTL) + refresh tokens (30-day, rotated on use, revoked on logout). Type claim (access/refresh/totp_pending) prevents token misuse.
2FA / TOTP	pyotp TOTP (RFC 6238). Setup flow: /totp/setup → QR scan → /totp/enable (first code verify). Login step-2 via short-lived totp_pending JWT (5-min TTL). valid_window=1 allows ±30s clock skew.
Desktop Credentials	Anthropic API key + iRacing credentials stored exclusively in OS keyring (Windows Credential Manager / macOS Keychain). Config file (~/.iracing_setup_advisor.json) never contains secrets.
Rate Limiting	Redis sorted-set sliding window per user. Different limits per tier (free: tighter, team: relaxed). AI quota checked before stream, incremented after.
Audit Logging	Immutable AuditLog table records: login success/failure, TOTP events, session uploads/deletes, billing events, admin actions, leaderboard submissions. Never deleted. IP + User-Agent captured.
Object Storage Access	All S3/R2 objects use presigned URLs (GET: 1-hour TTL, PUT: 15-min TTL). No public bucket. Keys namespaced by user_id. GDPR erasure via delete_user_objects().
CORS	Strict allowlist of origins in Settings.allowed_origins. Credentials=True for cookie-based flows.
Stripe Webhooks	Webhook signature validated with stripe.Webhook.construct_event(). Subscription status only updated via webhook, never client-side.

8. External Integrations

Service	Library/Version	Usage	Credentials
Anthropic Claude API	claude-haiku-4-5-20251001 (default), claude-sonnet-4-6 (premium)	Three usage patterns: (1) Standard streaming via get_ai_recommendations_stream — builds comprehensive prompt from AnalysisReport + setup + style + sector data; (2) Steven persona — injects DriverProfile context into system prompt; (3) Telemetry Agent — tool-use loop, up to 6 rounds, tools executed locally.	ANTHROPIC_API_KEY (desktop: keyring; server: env var)
iRacing Member Data API	https://members-ng.iracing.com	Cookie-based auth (POST /auth with base64(SHA-256(password + email.lower()))). Most endpoints return {link: s3_url} — client follows S3 redirect. Lap times in tenths-of-milliseconds (÷10,000 for seconds). Rate limit: 240 req/min enforced by 250ms inter-request gap.	User email + password (OS keyring)
Stripe	stripe-python 11.x	Checkout sessions for pro_monthly/pro_annual/team_monthly/team_annual plans. 7-day free trial on pro. Customer Portal for self-serve billing management. Webhook events: customer.subscription.created/updated/deleted, invoice.payment_failed.	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_* (env vars)
AWS S3 / Cloudflare R2	boto3 1.35+	Optional custom endpoint_url for R2 compatibility. Key namespacing: ibt/{user_id}/{session_id}.ibt, reports/{user_id}/{session_id}.json, setups/{user_id}/{setup_id}.sto, exports/{user_id}/{session_id}.pdf. Presigned upload (PUT, 15-min) and download (GET, 1-hour) URLs.	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_BUCKET, S3_ENDPOINT_URL (env vars)
Redis	redis[hiredis] 5.x + ARQ 0.26	Rate limiting (sorted-set sliding window), ARQ job queue (iracing_advisor queue), WebSocket manager not using Redis (in-process), session caching.	REDIS_URL (env var)
PostgreSQL	SQLAlchemy 2.0 async + asyncpg	pool_size=10, pool_pre_ping=True. All models use UUID PKs. JSONB for flexible metadata (audit, setup params). Alembic for schema migrations.	DATABASE_URL (env var)

9. File Formats & Persistence

9.1 File Formats

Format	Origin	Parser/Writer	Notes
.ibt	iRacing simulator	IBTParser (core/ibt_parser.py)	Binary. 144-byte header + variable metadata + 60 Hz sample records. Channels as parallel NumPy arrays. Up to 500 MB.
.htm / .html	iRacing garage export	SetupParser / SetupExporter (core/setup_parser.py)	HTML tables. Parsed into SetupSection list + flat dict. Written back with same structure for garage import.
.sto	iRacing garage (native)	StoWriter (core/setup_parser.py)	YAML-like text: CarCode, TrackName, SetupName header + named sections. Default path: ~/Documents/iRacing/setups//.sto.
.json (reports)	Analysis pipeline output	json.dumps / jobs.py	Serialised AnalysisReport + all sub-reports. Stored in S3 under reports/ key. NumPy arrays converted via _safe() helper (tolist/item).
.json (profile)	race_engineer.py	DriverProfileManager.save/load	Atomic write via os.replace. Path: ~/.iracing_engineer_profile.json. Stores DriverProfile dataclass (asdict). Reconstructs nested WeaknessRecord etc.
.json (config)	Desktop app	core/config.py (load_cfg/save_cfg)	Path: ~/.iracing_setup_advisor.json. Never contains secrets. Stores: last_dir, geometry, theme, units, palette_recent, onboarding_dismissed.
.pdf (reports)	PDF export pipeline	core/pdf_report.py (ReportLab)	Generated by generate_pdf_report(). Embeds track map (LineCollection heatmap) and lap delta chart (fill_between) as RLImage floatables.
.csv	MoTeC / external export	core/motec_export.py	Channel data export in MoTeC i2 compatible format. Optional.

9.2 Local Persistence (Desktop)

Store	Contents	Location
OS Keyring	Anthropic API key, iRacing email + password, cloud access/refresh tokens	Windows Credential Manager / macOS Keychain
Config JSON	UI preferences, last directory, geometry, command palette recents, consent flags	~/.iracing_setup_advisor.json
Driver Profile JSON	DriverProfile: style EWAs, session history, weakness records, car class prefs	~/.iracing_engineer_profile.json
License Cache JSON	Tier, features, last-verified timestamp (offline fallback)	~/.iracing_setup_advisor_license.json
App Logs	Rotating file handler 5 MB × 3 backups. Crash reports with traceback.	~/.iracing_setup_advisor_logs/app.log
IBT Cache	Parsed TelemetryData cached to avoid re-parsing. LRU eviction at 20 entries.	In-memory (evicted on app restart)

10. Deployment & Infrastructure

10.1 Docker Compose (docker-compose.yml)

Service	Image	Port	Notes
postgres	postgres:16	5432	Primary data store. Persistent volume.
redis	redis:7-alpine	6379	Job queue + rate limiting. Persistent volume (AOF).
api	Dockerfile (python:3.12-slim)	8000	FastAPI + Uvicorn. 4 workers in prod. Mounts files/ for shared analysis pipeline.
worker	Same Dockerfile	—	ARQ worker. Same image, different CMD: arq backend.jobs.WorkerSettings. max_jobs=4, job_timeout=300s.
minio	minio/minio (local dev only)	9000/9001	Local S3-compatible object storage for development. Not deployed in production.

10.2 Environment Variables (.env.example)

Variable	Purpose
SECRET_KEY	JWT signing key (min 32 chars random)
DATABASE_URL	postgresql+asyncpg://user:pass@host:5432/dbname
REDIS_URL	redis://localhost:6379/0
ANTHROPIC_API_KEY	Server-side Claude API key for cloud AI routes
STRIPE_SECRET_KEY	Stripe API key (sk_live_...)
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret (whsec_...)
STRIPE_PRICE_PRO_MONTHLY / ANNUAL	Stripe Price IDs for pro plan
STRIPE_PRICE_TEAM_MONTHLY / ANNUAL	Stripe Price IDs for team plan
AWS_ACCESS_KEY_ID / SECRET_ACCESS_KEY	S3 or Cloudflare R2 credentials
S3_BUCKET	Bucket name for all object storage
S3_ENDPOINT_URL	Optional: Cloudflare R2 endpoint URL
FRONTEND_URL	Base URL for email links (e.g. https://app.iracing-advisor.com)
ALLOWED_ORIGINS	CORS whitelist (comma-separated)
DEBUG	true|false — enables /docs, /redoc, verbose logging
ENVIRONMENT	development|staging|production

10.3 Desktop App Distribution

- Entry point: **files/main.py** (App().mainloop()). Packaged with PyInstaller into a single-folder bundle.
- Requires Python 3.10+. Optional dependencies: pyirsdk (live telemetry), tkinterdnd2 (drag & drop), websocket-client (cloud sync).
- App icon: files/app.ico. Version/copyright: files/version.py.
- All core analysis modules are pure Python + NumPy — no C extensions beyond NumPy/Matplotlib.

• Geometry (window size/position) and last-active tab are saved to config on close and restored on launch.

11. Key Algorithms & Design Decisions

EWA Driver Profile Updates

All style traits and balance metrics use Exponential Weighted Average (α=0.20): new_value = α×new + (1–α)×old. When old == 0.0 (first session), returns new directly to avoid initial zero-bias. Weakness severity uses α=0.30 (faster response). Corner type classification: <30 km/h = hairpin, <55 = slow, <85 = medium, ≥85 = fast.

Fuzzy Command Palette Scoring

Score hierarchy (highest wins): exact match (10.0) → prefix match (8.0) → substring (6.0) → any word starts with query (4.0) → difflib SequenceMatcher ratio > 0.4 (ratio value, else 0.0). Top 20 results shown. Recently-used list (8 items) stored in config.

iRacing API Auth

Password hash = base64(SHA-256(password + email.lower())). Posted to /auth endpoint → returns authtoken_members cookie. Subsequent requests carry cookie. Most endpoints return {link: S3_URL} and client follows S3 redirect automatically.

Lap Delta Computation

LapDeltaAnalyzer resamples both laps to a common LapDistPct grid (0.0–1.0, 1000 points). Computes per-position time delta via cumulative time integral over speed channel. Returns dist_pct[] + delta_s[] arrays used in both PDF charts and OBS overlay live interpolation.

IBT Lap Boundary Detection

Lap boundaries are detected from SessionNum and LapCompleted channels. A new lap starts when LapCompleted increments. Sample indices (start, end) are stored in lap_boundaries[], enabling O(1) slicing of any channel for any lap.

Sector Analysis

Track divided into N equal LapDistPct segments. For each segment, per-lap crossing times are computed via linear interpolation at the boundary sample. Sector consistency = 1 − CV (coefficient of variation). Theoretical best = sum of each sector's minimum time across all laps.

ML Lap Time Prediction

NumPy Ridge Regression (no sklearn dependency). Features: 8-element vector (balance score, front/rear avg tire temps, air/track temp, fuel/lap, avg grip utilisation, lap count). Trained on all loaded sessions. Confidence = 1 − (residual_std / mean_lap_time), clamped to [0, 1].

Telemetry Agent Tool-Use

Claude is given 5 tool declarations. In each round, the model returns text and/or tool_use blocks. Tool results are fed back as user messages. Loop continues for up to MAX_TOOL_ROUNDS=6 iterations or until stop_reason=end_turn. All tool execution is local — only question text + JSON results cross the API boundary.

12. Dependency Summary

12.1 Desktop App (requirements.txt)

Package	Version	Purpose
customtkinter	≥ 5.2	Dark-mode GUI framework
matplotlib	3.9.3	All charts and visualisations
numpy	1.26.4	Numerical computation throughout analysis pipeline
reportlab	4.2.5	PDF report generation
scipy	1.14.1	Signal processing (optional, fallback if missing)
anthropic	0.40.0	Claude API client (streaming)
keyring	latest	OS credential store (Windows/macOS/Linux)
requests	2.32.3	iRacing API + cloud sync HTTP calls
websocket-client	1.8.0	WebSocket session status watcher (cloud sync)
tkinterdnd2	optional	Drag-and-drop .ibt / .htm file support
pyirsdk	optional	iRacing shared memory for live telemetry

12.2 Backend (requirements-backend.txt)

Package	Version	Purpose
fastapi	0.115.5	Async web framework + OpenAPI
uvicorn[standard]	0.32.1	ASGI server (+ websockets extra)
gunicorn	23.0.0	Process manager for production
sqlalchemy[asyncio]	2.0.36	Async ORM
asyncpg	0.30.0	Async PostgreSQL driver
alembic	1.14.0	Database migrations
python-jose[cryptography]	3.3.0	JWT encode/decode
passlib[bcrypt]	1.7.4	Password hashing
pydantic[email]	2.10.3	Request/response validation
pydantic-settings	2.6.1	Environment variable settings
redis[hiredis]	5.2.1	Redis client + hiredis parser
arq	0.26.1	Async Redis job queue
boto3	1.35.84	S3 / Cloudflare R2 object storage
stripe	11.3.0	Stripe payments + webhooks
anthropic	0.40.0	Claude API (server-side)
pyotp	2.9.0	TOTP 2FA generation/verification
qrcode[pil]	8.0	QR code PNG generation for TOTP setup
websocket-client	1.8.0	Desktop WS status client

13. Current Build Status & Roadmap

13.1 Completed Features

- ✓ IBT parser, analysis engine, 15+ chart types, sector/corner/style/consistency analysis
- ✓ Setup parser (.htm/.sto), tech inspector, setup impact predictor, optimizer
- ✓ AI Setup Recommendations + Q&A; chat (streaming SSE, Claude Haiku)
- ✓ Race Engineer 'Steven' persona (persistent EWA driver profile, 3-session unlock)
- ✓ Natural Language Telemetry Agent (Claude tool-use, all execution local)
- ✓ Session Evolution AI Summary (per car+track historical comparison)
- ✓ iRacing Data API integration (iRating, SR, results, best laps, correlation)
- ✓ iRacing tab: stats, trend charts, results table, personal bests, best lap trend chart
- ✓ Community Leaderboard (opt-in, verified, per car+track, rank/percentile)
- ✓ OBS Overlay HUD (frameless, always-on-top, live delta sparkline, tyre temps)
- ✓ PDF Report generation (ReportLab, embedded track map + lap delta charts)
- ✓ Full SaaS Backend: FastAPI, PostgreSQL, Redis, ARQ, S3/R2, Stripe, JWT
- ✓ 2FA / TOTP: setup flow, QR code, step-2 login, enable/disable
- ✓ Audit Log: immutable events table, admin query endpoint
- ✓ WebSocket job status: session processing updates, user-wide event stream
- ✓ Command Palette: fuzzy scoring, recently used, keyboard navigation
- ✓ .sto setup file writer (native iRacing format, auto-paths to setups folder)
- ✓ File watcher (auto-load new .ibt), drag-and-drop, batch load
- ✓ ML lap time predictor, race strategy calculator, stint planner
- ✓ First-run onboarding banner, OS keyring credential storage
- ✓ Docker Compose deployment (API + Worker + Postgres + Redis + MinIO)

13.2 Potential Future Enhancements

- ■ Discord / Google OAuth login integration
- ■ Mobile companion app (React Native PWA)
- ■ Natural language setup search ('softer rear for Monza')
- ■ WebRTC peer-to-peer screen share for coach sessions
- ■ Automated iRacing series calendar integration (upcoming events, base setups)
- ■ Voice coaching mode (text-to-speech Steven debrief post-session)
- ■ Setup version history with git-like diff and rollback
- ■ Public API for third-party integrations (Discord bot, stream overlays)
- ■ GDPR data export / erasure endpoints
- ■ Multi-language localisation (French, German, Spanish)